

Lab 1 Pre-Lab: Building Programs with Functions

Objective

The goal of this pre-lab is to understand the fundamental components required to build a complete C program using functions. We will review how to define functions, handle user input, process data, and use conditional logic. The examples below will prepare you for this week's "Cinema Ticket Calculator" lab by demonstrating how to break a problem down into smaller, functional pieces.

Core Concepts for This Week

Before looking at the examples, let's review the essential tools we'll be using:

1. **Including Libraries:** A C program often starts with `#include <stdio.h>`.

- `std` stands for **standard**.
- `io` stands for **input-output**.
- `.h` indicates a **header file**, which allows us to use pre-written functions like `printf()` and `scanf()`.

2. **Defining a Function:** The basic structure of a function is:

```
return_type function_name(parameter1_type parameter1_name, ...) {  
    // Code to perform a task...  
    return value; // This value must match the return_type  
}
```

- The compiler knows it's a function because of the parentheses `()`.
- The function's code is contained within curly braces `{}`.
- If a function doesn't return any value, its `return_type` is `void`.

3. **Data Types:** Every variable has a type. The most common are:

- `int`: For whole numbers (e.g., `10`, `150`).
- `float`: For numbers with decimal points (e.g., `95.5`, `1.75`).
- `char`: For a single character (e.g., `'A'`, `'B'`).

4. **Input and Output:**

- `printf()`: Prints formatted text to the console.
- `scanf()`: Reads formatted input from the user. It needs two things:
 1. A **format specifier** to know what type of data to expect (`%d` for int, `%f` for float, `%s` for string).
 2. The memory address of the variable to store the data in. We use the `&` symbol for this (e.g., `&midterm_grade`).

Example 1: Calculate Student's Final Grade

Let's build a program that takes a student's midterm and final grades, calculates the final point, and determines the letter grade.

Function 1: `calculate_final_point`

This function calculates the final point based on weighted averages.

- **Description:** The final point is calculated as 40% of the midterm and 60% of the final.
- **Parameters:** `int midterm`, `int final`.
- **Returns:** `float final_point`.

```
float calculate_final_point(int midterm, int final) {  
    // The calculation returns a float value  
    return midterm * 0.4 + final * 0.6;  
}
```

Function 2: `calculate_letter_grade`

This function determines the letter grade from a final point using conditional logic.

- **Description:** Returns a letter grade ('A', 'B', etc.) based on the score.
- **Parameters:** `float final_point`.
- **Returns:** `char letter_grade`.

```
char calculate_letter_grade(float final_point) {  
    if (final_point >= 85 && final_point <= 100) {  
        return 'A';  
    } else if (final_point >= 75) { // No need for final_point < 85, it's  
implied  
        return 'B';  
    } else if (final_point >= 60) {  
        return 'C';  
    } else if (final_point >= 50) {  
        return 'D';  
    } else {  
        return 'F';  
    }  
}
```

Note on if/else: Conditional statements like `if` and `else if` are essential for making decisions in code. While we are learning them here, remember that conditional structures are strictly forbidden in your Labwork 1 assignment. Your lab will focus only on a sequence of calculations.

The `main` Function: Putting It All Together

The `main` function is the entry point of our program. Without it, the code won't run. It orchestrates the calls to our other functions.

```
int main() {
    int midterm_grade;
    int final_grade;

    printf("Please enter your midterm grade: ");
    scanf("%d", &midterm_grade);

    printf("Please enter your final grade: ");
    scanf("%d", &final_grade);

    // 1. Call the first function to get the final point
    float point = calculate_final_point(midterm_grade, final_grade);

    // 2. Use the result of the first function as input for the second one
    char grade = calculate_letter_grade(point);

    // 3. Display the final result to the user
    printf("Your final point is: %.2f\n", point);
    printf("Your letter grade is: %c\n", grade);

    return 0; // Indicates the program finished successfully
}
```

How it works: This example shows a common pattern:

1. Get input from the user in `main`.
2. Call a function to perform a calculation (`calculate_final_point`).
3. Use the returned value as input for another function (`calculate_letter_grade`).
4. Display the final result.

Your Cinema Ticket lab will follow this exact pattern. You will get inputs, call a series of calculation functions, and use the return value of one as the input for the next, until you have the final bill.